

Lista 7: Zaawansowane Grupowanie Danych

Uczenie Maszynowe – Uniwersytet Wrocławski

Semestr zimowy 2024/2025

Uwaga: Każde zadanie jest warte 2 punkty. Rozwiązania powinny zawierać kod w Pythonie oraz krótkie komentarze wyjaśniające kluczowe decyzje implementacyjne.

Zadanie 1: Semi-supervised Learning z Klasteryzacją (2 pkt)

Scenariusz: Firma “WroMed” gromadzi dane medyczne pacjentów, ale etykietowanie chorób przez lekarzy jest kosztowne i czasochłonne. Masz do dyspozycji duży zbiór nieetykietowanych danych i niewielki podzbiór (5%) z etykietami. Twoim zadaniem jest wykorzystanie klasteryzacji do propagacji etykiet na pozostałe dane.

Dane: Użyj zbioru `digits` ze `sklearn.datasets` (1797 obrazów cyfr 8x8, 10 klas).

Polecenia:

- (a) **Symulacja braku etykiet:** Losowo ukryj 95% etykiet (oznacz jako -1). Zachowaj tylko 5% danych jako “etykietowane”.
- (b) **Klasteryzacja + propagacja:** Wykonaj klasteryzację K-Means ($k = 10$). Dla każdego klastra znajdź najczęstszą etykietę wśród etykietowanych punktów i przypisz ją wszystkim punktom w klastrze.
- (c) **Porównanie metod:** Zaimplementuj dwie strategie propagacji:
 - **Głosowanie:** Najczęstsza etykieta w klastrze (punkt b)
 - **Najbliższy centroid:** Dla klastrów bez etykiet, przypisz etykietę najbliższego klastra z etykietami
- (d) **Ewaluacja:** Oblicz `accuracy` na **nieetykietowanych** danych. Porównaj z:
 - k-NN ($k = 5$) trenowanym tylko na etykietowanych danych
 - Pełny nadzorowany klasyfikator (100% etykiet) – górna granica

Jak zmienia się `accuracy` w zależności od % dostępnych etykiet (1%, 5%, 10%, 20%)?

Zadanie 2: Anomalie w Sieci Czujników (2 pkt)

Scenariusz: Firma “WroExpress” monitoruje flotę 200 tramwajów za pomocą czujników temperatury, wibracji i prędkości. Po serii awarii, Twoje zadanie to wykrycie anomalnych zachowań tramwajów przed ich całkowitą awarią.

Dane: Skorzystaj z syntetycznych danych generowanych w kodzie startowym lub z własnych pomiarów symulujących zachowanie czujników.

Polecenia:

- (a) **Metoda K-Means:** Zaimplementuj wykrywanie anomalii oparte na odległości od centroidów. Punkt jest anomalią, jeśli jego odległość od najbliższego centroidu przekracza próg (np. 95. percentyl).
- (b) **Metoda DBSCAN:** Użyj punktów oznaczonych jako szum (label = -1) przez DBSCAN jako kandydatów na anomalie. Dobierz parametry `eps` i `min_samples`.
- (c) **Metoda LOF:** Zastosuj Local Outlier Factor i porównaj wyniki z poprzednimi metodami.
- (d) **Analiza porównawcza:** Dla każdej metody oblicz precision, recall i F1-score (zakładając, że znasz prawdziwe anomalie z danych testowych). Która metoda najlepiej wykrywa anomalie w tym scenariuszu? Uzasadnij.

Kod startowy:

```
import numpy as np
from sklearn.cluster import KMeans, DBSCAN
from sklearn.neighbors import LocalOutlierFactor

def generate_sensor_data(n_normal=180, n_anomaly=20, random_state=42):
    """Generuje dane czujników z anomaliami."""
    np.random.seed(random_state)

    # Normalne tramwaje - 3 klastry operacyjne
    normal = np.vstack([
        np.random.randn(60, 3) * 0.5 + [50, 10, 30], # temp, vibration,
        np.random.randn(60, 3) * 0.5 + [55, 12, 35], speed
        np.random.randn(60, 3) * 0.5 + [48, 8, 28],
    ])

    # Anomalie - nietypowe zachowania
    anomalies = np.vstack([
        np.random.randn(10, 3) * 0.3 + [80, 25, 15], # przegrzanie
        np.random.randn(10, 3) * 0.3 + [30, 30, 5], # awaria silnika
    ])

    X = np.vstack([normal, anomalies])
    y_true = np.array([0]*n_normal + [1]*n_anomaly) # 0=normal, 1=
        anomaly

    return X, y_true

def detect_anomalies_kmeans(X, k=3, percentile=95):
    """Wykrywanie anomalii metoda K-Means."""
    # TODO: Zaimplementuj
    pass
```

```
def detect_anomalies_dbscan(X, eps=5, min_samples=5):  
    """Wykrywanie anomalii metoda DBSCAN."""  
    # TODO: Zaimplementuj  
    pass
```

Zadanie 3: Klasteryzacja Trajektorii GPS (2 pkt)

Scenariusz: Miasto Wrocław analizuje wzorce mobilności rowerzystów na podstawie danych z systemu “WroVelo”. Trajektorie mają różne długości i prędkości, więc standardowa odległość euklidesowa nie działa.

Dane: Skorzystaj z syntetycznych trajektorii generowanych w kodzie startowym lub z publicznych zbiorów danych GPS (np. GeoLife, OpenStreetMap traces).

Polecenia:

- Implementacja DTW:** Zaimplementuj algorytm Dynamic Time Warping do obliczania odległości między dwiema trajektoriami. Możesz użyć biblioteki `dtaidistance` lub napisać własną implementację.
- Macierz odległości:** Oblicz macierz odległości DTW dla wszystkich par trajektorii. Zwróć uwagę na złożoność obliczeniową.
- Hierarchiczne grupowanie (Ward):** Zastosuj metodę Warda z prekalkulowaną macierzą odległości. Zwizualizuj dendrogram i wybierz odpowiednią liczbę klastrów.
- Interpretacja:** Dla każdego klastra narysuj reprezentatywne trajektorie. Jakie wzorce mobilności odkryłeś? (np. trasy do pracy, rekreacyjne, turystyczne)

Kod startowy:

```
import numpy as np  
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster  
from scipy.spatial.distance import squareform  
import matplotlib.pyplot as plt  
  
def generate_trajectories(n_trajectories=50, random_state=42):  
    """Generuje syntetyczne trajektorie GPS."""  
    np.random.seed(random_state)  
    trajectories = []  
  
    # Typ 1: Trasa do centrum (z roznych stron)  
    for _ in range(15):  
        length = np.random.randint(20, 40)  
        start = np.random.randn(2) * 5 + [10, 10]  
        end = np.array([0, 0]) # centrum  
        t = np.linspace(0, 1, length).reshape(-1, 1)  
        traj = start + t * (end - start) + np.random.randn(length, 2) *  
            0.3  
        trajectories.append(traj)  
  
    # Typ 2: Trasa rekreacyjna (petla)  
    for _ in range(15):
```

```

        length = np.random.randint(30, 50)
        t = np.linspace(0, 2*np.pi, length)
        r = 3 + np.random.rand() * 2
        traj = np.column_stack([r*np.cos(t), r*np.sin(t)])
        traj += np.random.randn(length, 2) * 0.2
        trajectories.append(traj)

# Typ 3: Trasa wzdluz rzeki (liniowa)
for _ in range(20):
    length = np.random.randint(25, 45)
    start_x = np.random.rand() * 10 - 5
    traj = np.column_stack([
        np.linspace(start_x, start_x + 8, length),
        np.sin(np.linspace(0, np.pi, length)) * 2
    ])
    traj += np.random.randn(length, 2) * 0.2
    trajectories.append(traj)

return trajectories

def dtw_distance(traj1, traj2):
    """Oblicza odleglosc DTW miedzy dwiema trajektoriami."""
    # TODO: Zaimplementuj algorytm DTW
    pass

def compute_dtw_matrix(trajectories):
    """Oblicza macierz odleglosci DTW."""
    n = len(trajectories)
    D = np.zeros((n, n))
    # TODO: Wypelnij macierz (symetryczna!)
    return D

```

Zadanie 4: WroShop - Przetwarzanie Danych na Duzej Skali (2 pkt)

Scenariusz: Firma “WroShop” prowadzi platformę e-commerce we Wrocławiu. Ich dane transakcyjne rosną wykładniczo i przekraczają możliwości pandas. Twoim zadaniem jest poznanie narzędzi do przetwarzania dużych danych: Dask i PySpark.

Dane:

- data/wroshop_small.csv – 100,000 transakcji (pojedynczy plik)
- data/wroshop_medium/ – 1,000,000 transakcji (10 plików partycjonowanych)

Polecenia:

- Pandas baseline (100K):** Wczytaj małe dane i wykonaj operacje: groupby po kategorii, top 10 miast, filtrowanie. Zmierz czas i użycie pamięci.
- Pandas na większych danych (1M):** Wczytaj wszystkie pliki z wroshop_medium/ używając glob i pd.concat. Powtórz operacje. Zaobserwuj spowolnienie.
- Dask DataFrame:** Użyj dask.dataframe:

```
import dask.dataframe as dd
ddf = dd.read_csv('data/wroshop_medium/*.csv')
# Operacje sa LAZY - wykonaj .compute() na koncu
result = ddf.groupby('category')['total_amount'].mean().compute()
```

Zwizualizuj graf zadań. Porównaj czas z pandas.

- (d) **PySpark DataFrame:** Użyj pyspark (lub Google Colab):

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("WroShop").getOrCreate()
df = spark.read.csv('data/wroshop_medium/', header=True,
                    inferSchema=True)
df.groupby('category').avg('total_amount').show()
```

Pokaż plan wykonania: `result.explain()`.

- (e) **Porównanie:** Stwórz tabelę porównawczą czasów. Kiedy używać pandas/Dask/Spark?

Zadanie 5: WroAI - Mnożenie Macierzy w Praktyce ML (2 pkt)

Scenariusz: Startup “WroAI” buduje systemy uczenia maszynowego. Jako inżynier ML, Twoje zadanie to zoptymalizowanie kluczowych operacji obliczeniowych poprzez efektywne wykorzystanie mnożenia macierzy zamiast pętli.

Polecenia:

- (a) **Softmax Attention:** Zaimplementuj mechanizm uwagi z Transformerów:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Porównaj implementację z 3 zagnieżdżonymi pętlami vs wersję macierzową. Benchmark dla sekwencji długości 64, 256, 512.

- (b) **Macierz pomyłek bez pętli:** Zaimplementuj obliczanie confusion matrix używając mnożenia macierzy: Benchmark dla 10K, 100K próbek i 10, 50, 100 klas.
- (c) **Batch Normalization:** Zaimplementuj normalizację wsadową: Benchmark dla batch=128, features=1024, 4096.
- (d) **PageRank:** Zaimplementuj algorytm PageRank jako iterację macierzową:

$$r_{t+1} = d \cdot M \cdot r_t + \frac{1-d}{n} \cdot \mathbf{1}$$

gdzie M to znormalizowana macierz przejścia. Porównaj z wersją z pętlami. Zwizualizuj zbieżność.

Wymagania: Dla każdej operacji: (1) implementacja naiwna, (2) wyprowadzenie macierzowe, (3) benchmark, (4) weryfikacja poprawności (`np.allclose`).

Wskazówki ogólne

- Zalecane biblioteki: `numpy`, `scipy`, `scikit-learn`, `matplotlib`, `networkx` (dla zadania 3), `dtaiDistance` lub `tslearn` (dla zadania 4).
- Wykresy powinny mieć **tytuły, etykiety osi i legendy**.
- Kod powinien być **czytelny** i zawierać komentarze wyjaśniające kluczowe kroki.